USER GUIDE

Agentic AI Evaluator & Dashboard

1. Introduction

The Agentic AI Evaluator is a Python-based tool that automatically measures two trustworthiness indicators in autonomous AI agents:

- **Hallucination Rate** – percentage of agent responses containing factually incorrect or fabricated information.
- **Tool Misuse Rate** – percentage of tool calls that are inappropriate for the user's request.

The tool supports logs from three open-source agent frameworks:

- **AgenticSeek**
- **SuperAGI**
- **AutoGPT** (via a standard-format log capture)

Evaluation is performed by a **local Llama 3.1 8B model** running through **Ollama**, so the entire process works offline and does not require paid cloud APIs. Results are displayed in an interactive **Streamlit dashboard**.

2. System Requirements

Requirement	Minimum
Operating System	Linux (WSL2 recommended), macOS, or Windows 10/11
Python	3.10 or higher
RAM	8 GB (16 GB recommended)
Disk Space	10 GB free
Software	Ollama, Python, Streamlit

---

3. Installation and Setup

3.1 Install Ollama

1. Download and install Ollama from https://ollama.com.
2. Pull the judge model:

[ollama pull llama3.1:8b]

3. Start Ollama in a separate terminal:

[OLLAMA_HOST=0.0.0.0:11434 ollama serve]

Keep this terminal open while using the evaluator.

3.2 Obtain the Project Files

Place the following files into a project folder, for example ~/agentic-ai-evaluator/:

- evaluate_agent.py – the command-line evaluator
- dashboard.py – the Streamlit dashboard
- generate_synthetic_logs.py – synthetic log generator (optional)
- record.sh – terminal log recorder (optional)

3.3 Set Up the Python Environment

Open a terminal and navigate to the project folder:

[cd ~/agentic-ai-evaluator]

Create and activate a virtual environment:

[python3 -m venv venv

source venv/bin/activate]

Install the required Python packages:

[pip install ollama streamlit pandas]

Verify that Ollama is reachable:

[python -c "import ollama; print(ollama.list())"]

3.4 Create the Evaluator Shortcut (Optional)

Create a wrapper script so you can run the evaluator from any folder:

[cat > ~/evaluate << 'EOF'

#!/bin/bash

~/agentic-ai-evaluator/venv/bin/python ~/agentic-ai-evaluator/evaluate_agent.py "$@"

EOF

chmod +x ~/evaluate

export PATH="$HOME:$PATH"]

Now you can use:

[~/evaluate <log_file>]

from any directory.

4. Preparing Log Files

4.1 Supported Formats

The evaluator automatically detects the file format based on its extension and content. The following formats are supported:

Format	File Extension	Description
JSONL	.jsonl	One JSON object per line with user_prompt, agent_response, tool_calls
CSV	.csv	Standard CSV with the same columns
JSON	.json	JSON array of record objects
AutoGPT Plain Text	.log, .txt	Terminal log with ➤➤➤ delimiters and [Thought]/[Action] blocks

Format	File Extension	Description
SuperAGI DB Export	.csv	CSV exported from SuperAGI's agent_execution_feeds table
Real AutoGPT Terminal	.log	Full terminal capture from a real AutoGPT session

4.2 Capturing Logs from AgenticSeek

Use the script command to record a terminal session. First, navigate to your AgenticSeek installation and activate its environment. Then:

[script ~/agenticseeksession.log

python cli.py run –debug]

Type your question when prompted. After the agent finishes, type exit twice to stop recording. The log file will be saved at ~/agenticseeksession.log.

4.3 Exporting Logs from SuperAGI

1. Ensure SuperAGI is running via Docker Compose.
2. Run an agent through the SuperAGI web interface.
3. Find the latest execution ID:

bash

docker exec -it superagi-super__postgres-1 psql -U superagi -d super_agi_main -c \

"SELECT id, name, status, created_at FROM agent_executions ORDER BY created_at DESC LIMIT 1;"

4. Export the feed to CSV:

[docker exec -it superagi-super__postgres-1 psql -U superagi -d super_agi_main -c \

"\copy (SELECT role, feed, created_at FROM agent_execution_feeds WHERE agent_execution_id = <ID> ORDER BY created_at) TO '/tmp/feeds.csv' WITH CSV HEADER"

docker cp superagi-super__postgres-1:/tmp/feeds.csv ~/superagi_feeds.csv]

Replace <ID> with the execution ID from step 3.

4.4 AutoGPT Logs

Because the official AutoGPT parser is incompatible with many local models, complete AutoGPT terminal logs may crash the agent. The recommended approach is to capture the model's answer directly in AutoGPT format:

[echo "➤➤➤ What is the capital of Indonesia?" > autogpt_demo.log

echo "[Thought] I need to answer the question." >> autogpt_demo.log

echo "[Action] web_search" >> autogpt_demo.log

ollama run llama3.1:8b "Answer in one sentence: What is the capital of Indonesia?" | xargs -I {} echo "[Value] {}" >> autogpt_demo.log

echo "[Observation] Got answer." >> autogpt_demo.log]

This produces a log that the evaluator can process.

5. Using the Command-Line Evaluator

5.1 Basic Evaluation

To evaluate any supported log file, run:

[~/evaluate <path_to_log_file>]

The evaluator will automatically:

- Detect the file format
- Detect the agent type
- Load the appropriate allowed tools
- Run the LLM judge on each record
- Display the KPI summary

Example:

[~/evaluate superagi_feeds.csv]

5.2 Viewing Detailed Results

When prompted:

[Would you like to see individual log evaluations? (Y/N):]

Type Y and press Enter to see each record with the judge's verdict.

5.3 Saving Results to CSV

Use the --output option:

[~/evaluate superagi_feeds.csv --output results.csv]

5.4 Listing Available Logs

To see all log files in the project folder:

[~/evaluate –list]

5.5 Cleaning a Log File

If you only want to extract and clean the conversations without running the judge:

[~/evaluate <log_file> --clean --clean-output cleaned.json]

6. Using the Dashboard

6.1 Starting the Dashboard

Make sure Ollama is running in a separate terminal, then:

[cd ~/agentic-ai-evaluator

source venv/bin/activate

streamlit run dashboard.py]

Open your browser to http://localhost:8501.

6.2 Selecting a Log File

- Use the dropdown in the sidebar to select a log from the project folder, **or**
- Click the “Browse files” button to upload a new file.

6.3 Running the Evaluation

Click the **Run Evaluation** button. A progress bar will show processing status.

6.4 Understanding the Dashboard

The dashboard displays:

- **Agent Type** – the automatically detected framework
- **Total Records** – number of interactions evaluated
- **Hallucination Rate** – percentage of responses flagged as hallucinated
- **Tool Misuse Rate** – percentage of tool calls flagged as inappropriate

Below the metrics is a **Per-Record Details** table. You can expand each record to see the full prompt, response, tool, and verdict.

6.5 Exporting Results

Click **Download Summary as CSV** to save the evaluation summary. The file will be downloaded to your computer’s default Downloads folder.

7. Understanding the Results

KPI	Meaning	Good (Green)	Moderate (Yellow)	Poor (Red)
Hallucination Rate	How often the agent gave factually wrong or made-up answers	< 5%	5–15%	> 15%
Tool Misuse Rate	How often the agent used a tool inappropriately	< 10%	10–25%	> 25%

A low Hallucination Rate indicates the agent’s responses were generally accurate.
A low Tool Misuse Rate indicates the agent selected and used tools correctly.

8. Troubleshooting

Problem	Solution
ModuleNotFoundError: No module named 'ollama'	Run pip install ollama inside the virtual environment.
Ollama connection error	Ensure Ollama is running: OLLAMA_HOST=0.0.0.0:11434 ollama serve.
Dashboard won't start	Ensure you are in the venv and in the project folder.
No records found	Check that the log file is in a supported format and contains agent conversations.
Judge results vary on repeated runs	Normal for the local Llama model; overall trends remain stable.
SuperAGI export fails	Verify the container is running and the table name is agent_execution_feeds.
AutoGPT crashes	Use the workaround in Section 4.4 to create an AutoGPT-format log from the local LLM output.

9. Quick Reference Commands

Action	Command
Start Ollama	OLLAMA_HOST=0.0.0.0:11434 ollama serve
Activate environment	source ~/agentic-ai-evaluator/venv/bin/activate
Run evaluator on a file	~/evaluate <file>

Action	Command
List available logs	~/evaluate --list
Start dashboard	streamlit run dashboard.py
Generate synthetic logs	python generate_synthetic_logs.py

This user guide provides everything needed to install, run, and interpret the Agentic AI Evaluator on a fresh system. The instructions assume only the provided source files are available and do not depend on any pre-existing project paths.